

UIL COMPUTER SCIENCE WRITTEN TEST

2026 STATE

MAY 2026

General Directions (Please read carefully!)

1. DO NOT OPEN THE EXAM UNTIL TOLD TO DO SO.
2. There are 40 questions on this contest exam. You will have 45 minutes to complete this contest.
3. All answers must be legibly written on the answer sheet provided. Indicate your answers in the appropriate blanks provided on the answer sheet. Clean erasures are necessary for accurate grading.
4. You may write on the test packet or any additional scratch paper provided by the contest director, but NOT on the answer sheet, which is reserved for answers only.
5. All questions have ONE and only ONE correct answer. There is a 2-point penalty for all incorrect answers.
6. Tests may not be turned in until 45 minutes have elapsed. If you finish the test before the end of the allotted time, remain at your seat and retain your test until told to do otherwise. You may use this time to check your answers.
7. If you are in the process of actually writing an answer when the signal to stop is given, you may finish writing that answer.
8. All provided code segments are intended to be syntactically correct, unless otherwise stated. You may also assume that any undefined variables are defined as used.
9. A reference to many commonly used Java classes is provided with the test, and you may use this reference sheet during the contest. AFTER THE CONTEST BEGINS, you may detach the reference sheet from the test booklet if you wish.
10. Assume that any necessary import statements for standard Java SE packages and classes (e.g., `java.util`, `System`, etc.) are included in any programs or code segments that refer to methods from these classes and packages.
11. NO CALCULATORS of any kind may be used during this contest.

Scoring

1. Correct answers will receive **6 points**.
2. Incorrect answers will lose **2 points**.
3. Unanswered questions will neither receive nor lose any points.
4. In the event of a tie, the student with the highest percentage of attempted questions correct shall win the tie.

STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

package java.lang

```
class Object
    boolean equals(Object anotherObject)
    String toString()
    int hashCode()

interface Comparable<T>
    int compareTo(T anotherObject)
        Returns a value < 0 if this is less than anotherObject.
        Returns a value = 0 if this is equal to anotherObject.
        Returns a value > 0 if this is greater than anotherObject.

class Integer implements Comparable<Integer>
    Integer(int value)
    int intValue()
    boolean equals(Object anotherObject)
    String toString()
    String toString(int i, int radix)
    int compareTo(Integer anotherInteger)
    static int parseInt(String s)

class Double implements Comparable<Double>
    Double(double value)
    double doubleValue()
    boolean equals(Object anotherObject)
    String toString()
    int compareTo(Double anotherDouble)
    static double parseDouble(String s)

class String implements Comparable<String>
    int compareTo(String anotherString)
    boolean equals(Object anotherObject)
    int length()
    String substring(int begin)
        Returns substring(begin, length()).
    String substring(int begin, int end)
        Returns the substring from index begin through index (end - 1).
    int indexOf(String str)
        Returns the index within this string of the first occurrence of str. Returns
        -1 if str is not found.
    int indexOf(String str, int fromIndex)
        Returns the index within this string of the first occurrence of str, starting
        the search at fromIndex. Returns -1 if str is not found.
    int indexOf(int ch)
    int indexOf(int ch, int fromIndex)
    char charAt(int index)
    String toLowerCase()
    String toUpperCase()
    String[] split(String regex)
    boolean matches(String regex)
    String replaceAll(String regex, String str)

class Character
    static boolean isDigit(char ch)
    static boolean isLetter(char ch)
    static boolean isLetterOrDigit(char ch)
    static boolean isLowerCase(char ch)
    static boolean isUpperCase(char ch)
    static char toUpperCase(char ch)
    static char toLowerCase(char ch)

class Math
    static int abs(int a)
    static double abs(double a)
    static double pow(double base, double exponent)
    static double sqrt(double a)
    static double ceil(double a)
    static double floor(double a)
    static double min(double a, double b)
    static double max(double a, double b)
    static int min(int a, int b)
    static int max(int a, int b)
    static long round(double a)
    static double random()
        Returns a double greater than or equal to 0.0 and less than 1.0.
```

package java.util

```
interface List<E>
class ArrayList<E> implements List<E>
    boolean add(E item)
    int size()
    Iterator<E> iterator()
    ListIterator<E> listIterator()
    E get(int index)
    E set(int index, E item)
    void add(int index, E item)
    E remove(int index)

class LinkedList<E> implements List<E>, Queue<E>
    void addFirst(E item)
    void addLast(E item)
    E getFirst()
    E getLast()
    E removeFirst()
    E removeLast()

class Stack<E>
    boolean isEmpty()
    E peek()
    E pop()
    E push(E item)

interface Queue<E>
class PriorityQueue<E>
    boolean add(E item)
    boolean isEmpty()
    E peek()
    E remove()

interface Set<E>
class HashSet<E> implements Set<E>
class TreeSet<E> implements Set<E>
    boolean add(E item)
    boolean contains(Object item)
    boolean remove(Object item)
    int size()
    Iterator<E> iterator()
    boolean addAll(Collection<? extends E> c)
    boolean removeAll(Collection<?> c)
    boolean retainAll(Collection<?> c)

interface Map<K,V>
class HashMap<K,V> implements Map<K,V>
class TreeMap<K,V> implements Map<K,V>
    Object put(K key, V value)
    V get(Object key)
    boolean containsKey(Object key)
    int size()
    Set<K> keySet()
    Set<Map.Entry<K, V>> entrySet()

interface Iterator<E>
    boolean hasNext()
    E next()
    void remove()

interface ListIterator<E> extends Iterator<E>
    void add(E item)
    void set(E item)

class Scanner
    Scanner(InputStream source)
    Scanner(String str)
    boolean hasNext()
    boolean hasNextInt()
    boolean hasNextDouble()
    String next()
    int nextInt()
    double nextDouble()
    String nextLine()
    Scanner useDelimiter(String regex)
```

STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

Package `java.util.function`

```
Interface BiConsumer<T,U>  
    void accept(T t, U u)  
  
Interface BiFunction<T,U,R>  
    R apply(T t, U u)  
  
Interface BiPredicate<T,U>  
    boolean test(T t, U u)  
  
Interface Consumer<T>  
    void accept(T t)  
  
Interface Function<T,R>  
    R apply(T t)  
  
Interface Predicate<T>  
    boolean test(T t)  
  
Interface Supplier<T>  
    T get()
```

UIL COMPUTER SCIENCE WRITTEN TEST – 2026 STATE

Note: Correct responses are based on **Java SE Development Kit 25 (JDK 25)** from Oracle, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 25 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used. **For all output statements, assume that the System class has been statically imported using: `import static java.lang.System.*;`**

Question 1

How many 1s are in the binary representation of the base 10 number 28,520,985?

- A) 12 B) 13 C) 11 D) 10 E) 14

Question 2

What is output by the code to the right?

- A) 400 B) 395 C) 416 D) 415 E) 410

```
int x = 123;
int y = 456;
out.println(x + (y % x) - x ^ y);
```

Question 3

What is output by the code to the right?

The next line is a ruler that can be used for your convenience.

01234567890123456789012345678901234567890

- A) [42|-003.14|Hello]
 B) [42 |-003.14|Hello]
 C) [42| -3.14|Hello]
 D) [42 | -3.14|Hello]
 E) [42 |-003.14|HelloWorld]

```
int x = 42;
double y = -3.14;
String s = "HelloWorld";

out.printf("[% -8d|%07.2f|%.5s]%n",
           x, y, s);
```

Question 4

What is output by the code to the right?

- A) true>trueHello World
 B) false>falseHello World
 C) true>falseHello World
 D) true>false[H,e,l,l,o, ,W,o,r,l,d]
 E) true>true[H,e,l,l,o, ,W,o,r,l,d]

```
String s = "Hello World";
char [] helloWorld = s.toCharArray();
String s2 =
    String.valueOf(helloWorld);

out.print(s.equals(s2));
out.print(s == s2);
out.print(helloWorld);
```

Question 5

What is output by the code to the right?

- A) false
 B) true

```
boolean a = true;
boolean b = false;
boolean c = a ^ b || a && b;
out.println(c && (a ^ c || b));
```

Question 6

What is output by the code to the right?

- A) 2147483647
 B) -2147483648
 C) There is no output due to a runtime error.
 D) There is no output due to a compile error.

```
int x = 0;
int count = 0;

while(x <= Integer.MAX_VALUE) {
    x = Math.incrementExact(x);
    count++;
}
out.println(count);
```

Question 7

What is output by the code to the right?

- A) 48 B) 191 C) 159 D) 253 E) 155

```
int a = 0b1010_1100;
int b = 0xF3;
int c = 073;
out.println(a & b ^ c | a >> 2 + 1);
```

Question 8 What is output by the code to the right? A) Zoo B) ZooWee C) ZooWeeMama D) Wee E) WeeMama	<pre>double d = 2.0; float f = 2.0f; double d2 = Double.NaN; float f2 = Float.NaN; if (d == f) out.print("Zoo"); if (d2 == f2) out.print("Wee"); if (d2 > f2) out.print("Mama");</pre>
Question 9 What is output by the code to the right? A) XYQ61 B) XZQ61 C) WXYQ63 D) XZ71 E) XZQ71	<pre>int a = 7, b = 2, c = 0; String r = ""; switch (a % 3 + b) { case 4: r += "W"; c = 2; case 3: r += "X"; if (c++ > 0 && --a > 5) r += "Y"; else r += "Z"; case 2: r += "Q"; break; default: r += "R"; } out.println(r + a + c);</pre>
Question 10 What is output by the code to the right? A) 0034 B) 7004 C) 4034 D) 7034 E) 4004	<pre>int[] a = {1, 4, 2, 8, 3}; int[] b = a; int[] c = new int[a.length]; for (int i = 0; i < a.length - 1; i++) c[i] = a[i + 1] - a[i]; b = c; a[0] += b[0] + c[0]; b[0] = 0; out.println(a[0] + " " + b[0] + c[0] + a[1]);</pre>
Question 11 What is output by the code to the right? A) 12 5678 B) 12 168 C) 12 34134 D) 12 345678 E) 46 345678	<pre>Scanner sc = new Scanner("12 34\n56 78"); int a = sc.nextInt(); String s = sc.nextLine(); Scanner sc2 = new Scanner(s + sc.nextInt() + sc.nextInt()); int x = 0; while (sc2.hasNextInt()) x += sc2.nextInt(); out.println(a + " " + x);</pre>

Question 12 What is output by the code to the right? A) 12 9 9 B) 9 12 9 C) 5 6 0 D) 12 6 9 E) 6 12 5	<pre>int s = 0, p = 1, t = 0; for (int i = 2; i <= 6; i++) { switch (i % 4) { case 0: s *= i; case 1: s += i; break; case 2: p *= i; case 3: t = s; s = p; p = t; break; } } out.println(s + " " + p + " " + t);</pre>
Question 13 What is output by the code to the right? A) 3 8 B) 3 4 C) 4 8 D) 2 4 E) 2 8	<pre>int a = 5, b = 3, c = 2; int d = ++a - b--; out.print(d + " "); out.print(a > b ? a + c : b * c);</pre>
Question 14 What is output by the code to the right? A) true B) false	<pre>out.println(Double.MIN_VALUE > 0);</pre>
Question 15 What is the largest possible sum of values in the list? A) 4 [127, 128, 127] [127, 128] B) 0 [127, 128, 127] [127, 128] C) 2 [127, 127] [127, 128] D) 2 [127, 128, 127] [127, 128] E) There is no output due to a runtime error.	<pre>ArrayList<Integer> a = new ArrayList<>(List.of(127, 128, 127, 128)); ArrayList<Integer> b = new ArrayList<>(List.of(127, 128, 127, 128)); int s = 0; for (int i = 0; i < a.size(); i++) if (a.get(i) == b.get(i)) s++; a.add(a.remove(0)); a.remove(Integer.valueOf(128)); b.subList(0, 2).clear(); out.println(s + " " + a + " " + b);</pre>
Question 16 What is output by the code to the right? A) 1213 B) 13 C) 123 D) There is no output due to a compile error. E) There is no output due to a runtime error.	<pre>try { String str = "12\n34"; Scanner s = new Scanner(str); out.print(s.nextLine()); } catch (IOException f) { out.print(1); } finally { out.print(3); }</pre>

Question 17 What is output by the line marked //q17 in the code to the right? A) [A, B, B, E, E, F, H, J, K, L, S] B) [A, B, E, J, F, K, S, H, L, E, B] C) [A, B, E, F, H, J, K, L, S] D) [A, B, E, J, F, K, S, H, L] E) There is no output due to a runtime error.	<pre> TreeMap<String,Integer> tm; tm = new TreeMap<>(); String[] s ="ABEJFKSHLEB".split(""); for(String r:s) { int num =(int)(Math.random() * 99); tm.put(r, num); } out.println(tm.keySet()); //q17 </pre>
Question 18 Assuming any error in the preceding code has been corrected, which of the following values could not possibly be output by the line marked //q18 in the code to the right? A) 0 B) 1 C) 98 D) 99 E) More than one of the above.	<pre> for(String h:tm.keySet()) out.print(tm.get(h)); //q18 </pre>
Question 19 Let N represent an arbitrary length of <code>String</code> s. In terms of N, what is the Big-$\mathcal{O}$ runtime of the code to the right (assume average-case)? A) $\mathcal{O}(N \log N)$ B) $\mathcal{O}(N)$ C) $\mathcal{O}(N^2)$ D) $\mathcal{O}(\log N)$ E) $\mathcal{O}(N^2 \log N)$	
Question 20 Which of the lines in the code to the right first causes an error (compile- or run-time)? A) //1 B) //2 C) //3 D) //4 E) None of the above. All lines are error-free.	<pre> int[][] m = new int[5][]; //1 m[3] = new int['A']; //2 out.print(m.hashCode()); //3 m[2][0] = 17; //4 </pre>
Question 21 What is output by the code to the right? A) 2570 B) 1400 C) 1825 D) There is no output due to a compile error. E) There is no output due to a runtime error.	<pre> String s = "30201"; int i = Integer.parseInt(s, 5); s = Integer.toString(i, 9); i = Integer.parseInt(s, 7); s = Integer.toString(i, 9); i = Integer.parseInt(s, 7); out.println(i); </pre>

Question 22 What can replace <1*> in the code to the right so that it compiles and runs without error? A) extends Fighter B) implements Fighter C) extends Wizard, Fighter D) extends Warrior E) More than one of the above.	<pre> class Fighter{ private int health; public Fighter() { health = 200; } public int getHealth() { return health; } public void setHealth(int h) { health = h; } public int attack(int eh) { return eh - 10; } } </pre>
Question 23 What can replace <2*> in the code to the right so that the Wizard() constructor initializes the health instance variable to 200, and the magic instance variable to the parameter m? A) super(); magic = m; B) magic = m; C) super(200); magic = m; D) super(200); E) A or B. F) A, B, or C. G) Any of the above. H) None of the above.	<pre> class Wizard <1*> { int magic; public Wizard(int m) { <2*>; } public int attack(int eh) { magic--; return eh - 20; } } </pre>
Question 24 Assuming <1*> and <2*> are filled in correctly, what is output by the line marked //q24 in the code to the right? A) 200 B) 180 C) 170 D) 160 E) There is no output due to a runtime error.	<pre> class Warrior <1*> { public int health; public Warrior() { health = 300; } public void heal(){ setHealth(health + 50); } } </pre>
Question 25 Assuming <1*> and <2*> are filled in correctly, what is output by the line marked //q25 in the code to the right? A) 260 23 B) 200 23 C) 200 25 D) 180 23 E) There is no output due to a runtime error.	<pre> ////////// client code ////////// Fighter f = new Fighter(); Wizard w1 = new Wizard(25); Warrior w2 = new Warrior(); int num = w1.attack(f.getHealth()); f.setHealth(num); num = w2.attack(f.getHealth()); f.setHealth(num); num = w1.attack(w2.getHealth()); w2.setHealth(num); num = f.attack(w2.getHealth()); w2.setHealth(num); w2.heal(); out.println(f.getHealth()); //q24 String o = w1.getHealth()+" "; out.println(o + w1.magic); //q25 out.println(w2.getHealth()); //q26 </pre>
Question 26 Assuming <1*> and <2*> are filled in correctly, what is output by the line marked //q26 in the code to the right? A) 320 B) 170 C) 250 D) 350 E) There is no output due to a runtime error.	<pre> ////////// client code ////////// Fighter f = new Fighter(); Wizard w1 = new Wizard(25); Warrior w2 = new Warrior(); int num = w1.attack(f.getHealth()); f.setHealth(num); num = w2.attack(f.getHealth()); f.setHealth(num); num = w1.attack(w2.getHealth()); w2.setHealth(num); num = f.attack(w2.getHealth()); w2.setHealth(num); w2.heal(); out.println(f.getHealth()); //q24 String o = w1.getHealth()+" "; out.println(o + w1.magic); //q25 out.println(w2.getHealth()); //q26 </pre>

Question 27 What is output by the line marked //q28 in the code to the right? A) false false B) false true C) true false D) true true E) There is no output due to a compile error.	<pre>String a = "3LF BOO On3 TW0 3E3"; String reg = "((\\d[A-Z][A-Z]) "; reg += "(\\w\\D\\S) ?){4,6}"; out.print(a.matches(reg)); out.print(" "); reg = ".+((?<=([e-x]))[2-8]).*"; reg += "\\s.*((?!\\s))\\d).*"; out.println(a.matches(reg));</pre>								
Question 28 What is output by the line marked //q28 in the code to the right? A) 14 B) 13 C) 22 D) 18 E) There is no output due to a runtime error.	<pre>public int recur(int n) { if(n <= 0) return 1; if(n % 3 == 1) return 2 * recur(n - 4); if(n < 10) return n + recur(n / 2); if(n % 7 < 2) return 3 + recur(n - 4); return recur(n - 6) + recur(n - 8); } //////// client code ////////// out.println(recur(13)); //q28 out.println(recur(43)); //q29</pre>								
Question 29 What is output by the line marked //q29 in the code to the right? A) 141 B) 128 C) 376 D) 484 E) There is no output due to a runtime error.									
Question 30 From which two sorting algorithms is the TIMSORT algorithm derived (commonly used in the Collections.sort() and Arrays.sort() methods)? A) SELECTIONSORT and INSERTIONSORT B) INSERTIONSORT and QUICKSORT C) SELECTIONSORT and QUICKSORT D) INSERTIONSORT and MERGESORT E) SELECTIONSORT and MERGESORT F) QUICKSORT and MERGESORT									
Question 31 Which of the following Java operators are right-to-left associative according to standard Java operator precedence rules? A) 1, 2, and 5 only. B) 2, 6, 7, and 8 only. C) 2, 3, 6, 7, and 8 only. D) 4, 6, 7, and 8 only. E) None of the above.	<table><tr><td>1. new</td><td>2. ++ (post-increment)</td></tr><tr><td>3. !=</td><td>4. ++ (pre-increment)</td></tr><tr><td>5. ==</td><td>6. =</td></tr><tr><td>7. <=</td><td>8. -> (lambda expression)</td></tr></table>	1. new	2. ++ (post-increment)	3. !=	4. ++ (pre-increment)	5. ==	6. =	7. <=	8. -> (lambda expression)
1. new	2. ++ (post-increment)								
3. !=	4. ++ (pre-increment)								
5. ==	6. =								
7. <=	8. -> (lambda expression)								
Question 32 What is output by the client code to the right? A) AB B) ABAB C) ABCA D) BCB C E) There is no output due to an error.	<pre>UnaryOperator<String> shift = s -> s.substring(1) + s.charAt(0); Predicate<String> p = s -> s.compareTo("BCA") >= 0; String r = Stream.iterate("ABC", shift) .limit(5) .filter(p.negate()) .map(s -> s.substring(0, 2)) .collect(Collectors.joining()); out.println(r);</pre>								

Question 33

What is output by the client code to the right?

- A) Beginner
true
MEDIUM
null
- B) EASY
true
2
null
- C) Beginner
false
MEDIUM
Exception
- D) Beginner
true
Intermediate
null
- E) There is no output due to an error.

```
enum Difficulty {
    EASY(1, "Beginner"),
    MEDIUM(2, "Intermediate"),
    HARD(3, "Advanced");

    private int level;
    private String description;

    Difficulty(int level, String description) {
        this.level = level;
        this.description = description;
    }

    public int getLevel() {
        return level;
    }

    public String getDescription() {
        return description;
    }

    public boolean isHarderThan(Difficulty other) {
        return this.level > other.level;
    }

    public static Difficulty fromLevel(int level) {
        for (Difficulty d : values()) {
            if (d.level == level) {
                return d;
            }
        }
        return null;
    }
}
```

Question 34

Which of the following statements about the Difficulty enum is true?

- A) Enums cannot have constructors.
- B) The values() method must be manually implemented.
- C) Each enum constant (EASY, MEDIUM, HARD) is an instance of Difficulty.
- D) Enum fields must be declared as static.
- E) More than one of the above is true.

Question 35

What would the following expression evaluate to?

```
Difficulty.MEDIUM.isHarderThan(Difficulty.HARD)
```

- A) true
- B) false
- C) There is no output due to a compile error.
- D) There is no output due to a runtime error.

```
////////// Client Code ///////////
Difficulty d1 = Difficulty.EASY;
Difficulty d2 = Difficulty.HARD;

out.println(d1.getDescription());
out.println(d2.isHarderThan(d1));
out.println(Difficulty.fromLevel(2));
out.println(Difficulty.fromLevel(4));
```

Question 36

What is output by the code to the right?

- A) +A+BX-A-B
- B) +A+BX-B-A
- C) +A-B+BX-A
- D) +B+AX-A-B
- E) The output cannot be determined until runtime.

```
static class R implements AutoCloseable {
    private String name;

    R(String n) {
        name = n;
        out.print("+" + name);
    }

    public void close() {
        out.print("-" + name);
    }
}

////////// Client Code ///////////
try (R a = new R("A"); R b = new R("B")) {
    out.print("X");
}
```

Question 37

What does the method `mysteryFunc1()` to the right compute?

- A) The sum of all elements in `nums`.
- B) The maximum element in `nums`.
- C) The maximum subarray sum of `nums`.
- D) The longest increasing subsequence in `nums`.
- E) A needlessly complicated way of returning the first element in `nums`.

Question 38

Suppose the method `mysteryFunc1()` was called with an array of n integers. What would be the time complexity of said method call?

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(\log n)$
- E) $O(n \log^2 n)$

```
private static class Tuple {
    int a, b, c, d;
}

private Tuple helper1(int[] arr, int l, int h) {
    if (l == h) {
        int val = arr[l];
        Tuple res = new Tuple();
        res.a = res.b = res.c = res.d = val;
        return res;
    }

    int m = (l + h) / 2;
    Tuple L = helper1(arr, l, m);
    Tuple R = helper1(arr, m + 1, h);
    return helper2(L, R);
}

private Tuple helper2(Tuple L, Tuple R) {
    Tuple res = new Tuple();
    res.a = L.a + R.a;
    res.b = Math.max(L.b, L.a + R.b);
    res.c = Math.max(R.c, L.c + R.a);
    res.d = Math.max(Math.max(L.d, R.d),
                      L.c + R.b);
    return res;
}

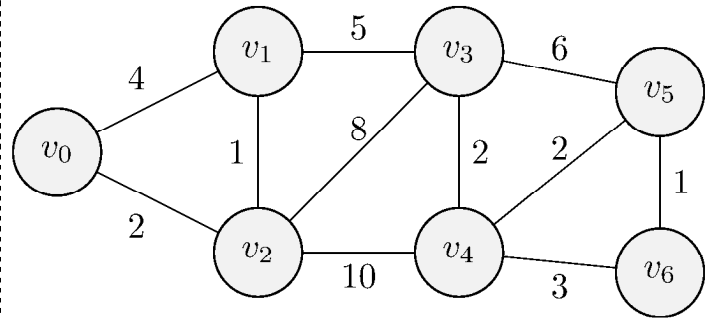
public int mysteryFunc1(int[] nums) {
    if (nums.length == 0) return 0;
    return helper1(nums, 0, nums.length - 1).d;
}
```

Question 39

Consider the weighted graph G to the right. What is the total weight of a minimum spanning tree of graph G ? Write your answer in the blank provided on your answer sheet.

Question 40

Consider the weighted graph G to the right. How many unique shortest path traversals exist starting from vertex v_0 to vertex v_6 ? Write your answer in the blank provided on your answer sheet.



★ ANSWER KEY – CONFIDENTIAL ★

UIL COMPUTER SCIENCE – 2026 STATE

Questions (+6 points for each correct answer, -2 points for each incorrect answer)

- | | | | |
|------------------|------------------|------------------|--------------------|
| 1) <u> A </u> | 11) <u> D </u> | 21) <u> E </u> | 31) <u> D </u> |
| 2) <u> D </u> | 12) <u> A </u> | 22) <u> A </u> | 32) <u> B </u> |
| 3) <u> B </u> | 13) <u> A </u> | 23) <u> E </u> | 33) <u> A </u> |
| 4) <u> C </u> | 14) <u> A </u> | 24) <u> C </u> | 34) <u> C </u> |
| 5) <u> A </u> | 15) <u> D </u> | 25) <u> B </u> | 35) <u> B </u> |
| 6) <u> C </u> | 16) <u> D </u> | 26) <u> D </u> | 36) <u> B </u> |
| 7) <u> C </u> | 17) <u> C </u> | 27) <u> D </u> | 37) <u> C </u> |
| 8) <u> A </u> | 18) <u> D </u> | 28) <u> C </u> | 38) <u> A </u> |
| 9) <u> E </u> | 19) <u> A </u> | 29) <u> D </u> | *39) <u> 13 </u> |
| 10) <u> B </u> | 20) <u> D </u> | 30) <u> D </u> | *40) <u> 2 </u> |

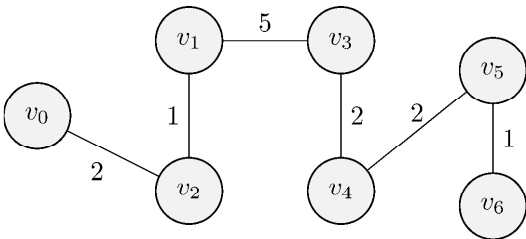
* See "Explanation" section below for alternate, acceptable answers.

Note: Correct responses are based on **Java SE Development Kit 25 (JDK 25)** from Sun Microsystems, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 25 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used.

Explanations:

1.	A	There is a trick to this where you can continuously do integer division on the number until it reaches 0, and count the number of odd values in the chain.
2.	D	Simple math.
3.	B	https://alvinalexander.com/programming/printf-format-cheat-sheet/
4.	C	<code>copyValueOf</code> makes a new string with the contents of the provided <code>CharSequence</code> . The contents are the same but the memory addresses are different. Character array <code>toString</code> prints as a normal <code>String</code> .
5.	A	Simple boolean logic.
6.	C	https://docs.oracle.com/javase/8/docs/api/java/lang/Math.html#incrementExact-int-
7.	C	First, decode the literals: <code>0b1010_1100</code> = 172, <code>0xF3</code> = 243, and 073 is octal (not decimal) = 59. Then, apply Java's precedence rules. The full chain from high to low is: <code>% * /</code> → <code>+ -</code> → <code><< >> >>></code> → <code>&</code> → <code>^</code> → <code> </code> . So the expression groups as <code>(a & b) ^ c (a >> (2 + 1))</code>
8.	A	If either operand is NaN, then the result of <code>==</code> is false https://docs.oracle.com/javase/specs/jls/se7/html/jls-15.html#jls-15.21.1
9.	E	First, <code>a % 3 + b = 7 % 3 + 2 = 1 + 2 = 3</code> , so we enter at case 3. case 3: <code>r</code> becomes "X". Now the <code>if: c++</code> is post-increment – <code>c</code> 's current value 0 is used in the comparison, <i>then</i> <code>c</code> becomes 1. Since <code>0 > 0</code> is false, the <code>&&</code> short-circuits and <code>--a</code> is never evaluated, so <code>a</code> stays 7. We take the <code>else</code> branch and <code>r</code> becomes "XZ". There is no <code>break</code> on case 3, so we fall through to case 2: <code>r</code> becomes "XZQ", then we hit the <code>break</code> . Finally, <code>r + a + c</code> is string concatenation: <code>"XZQ" + 7 + 1</code> → <code>"XZQ71"</code> .
10.	B	First, <code>b = a</code> makes <code>b</code> an alias of <code>a</code> . The loop builds a difference array in <code>c</code> : <code>c</code> becomes {3, -2, 6, -5, 0}. Note <code>c[4]</code> stays 0 because the loop runs to <code>a.length - 1</code> exclusive, so only indices 0-3 are written. Then <code>b = c</code> breaks <code>b</code> 's alias to <code>a</code> and makes <code>b</code> point at <code>c</code> instead. Now <code>b</code> and <code>c</code> reference the same object, while <code>a</code> is independent. <code>a[0] += b[0] + c[0]</code> : since <code>b</code> and <code>c</code> are the same array, <code>b[0]</code> and <code>c[0]</code> are both 3. So <code>a[0] = 1 + 3 + 3 = 7</code> . <code>b[0] = 0</code> : this also sets <code>c[0]</code> to 0 since they're the same array. <code>a</code> is unaffected. Final state: <code>a = {7, 4, 2, 8, 3}</code> , <code>b = c = {0, -2, 6, -5, 0}</code> . String concatenation: <code>"7" + "0" + "0" + "4"</code> → <code>"7004"</code> .
11.	D	<code>sc.nextInt()</code> reads 12 and stops – position is <i>before</i> the space. <code>sc.nextLine()</code> then consumes the rest of line 1: " 34". The remaining input is "56 78", so the two subsequent <code>nextInt()</code> calls return 56 and 78. Since <code>+</code> is left-associative and <code>s</code> is a <code>String</code> , the entire expression is string concatenation: <code>" 34" + 56 + 78</code> = <code>" 345678"</code> . The new <code>Scanner</code> reads that as one integer: 345678.
12.	A	<code>i=2: 2%4=2</code> . case 2: <code>p*=2</code> → <code>p=2</code> . Falls through to case 3: <code>t=0</code> , <code>s=2</code> , <code>p=0</code> . Break. <code>i=3: 3%4=3</code> . case 3: <code>t=2</code> , <code>s=0</code> , <code>p=2</code> . Break. <code>i=4: 4%4=0</code> . case 0: <code>s*=4</code> → 0. Falls through to case 1: <code>s+=4</code> → 4. Break. <code>i=5: 5%4=1</code> . case 1: <code>s+=5</code> → 9. Break. <code>i=6: 6%4=2</code> . case 2: <code>p*=6</code> → 12. Falls through to case 3: <code>t=9</code> , <code>s=12</code> , <code>p=9</code> . Break.
13.	A	<code>++a</code> pre-increments <code>a</code> to 6 and <code>b--</code> post-decrements <code>b</code> (using 3 in the expression, then dropping to 2), so <code>d = 6 - 3 = 3</code> , and since <code>a > b</code> (<code>6 > 2</code>) is true the ternary picks <code>a + c = 8</code> , giving <code>3 8</code> .

14.	A	https://docs.oracle.com/javase/8/docs/api/java/lang/Double.html#MIN_VALUE
15.	D	The <code>==</code> on <code>Integer</code> objects compares references, not values. Java caches <code>Integer</code> objects for values <code>-128</code> through <code>127</code>. So <code>a.get(0) == b.get(0)</code> compares two <code>Integer(127)</code> objects that are the same cached instance $\rightarrow$ <code>true</code>. But <code>a.get(1) == b.get(1)</code> compares two <code>Integer(128)</code> objects that are distinct heap allocations $\rightarrow$ <code>false</code>. Indices <code>0</code> and <code>2</code> match; <code>1</code> and <code>3</code> don't. <code>s = 2</code>. <code>a.remove(0)</code>: removes by index, returns <code>Integer 127</code>. <code>a = [128, 127, 128]</code>. <code>a.add(127)</code> appends it. <code>a = [128, 127, 128, 127]</code>. <code>a.remove(Integer.valueOf(128))</code>: removes by value — first occurrence only. <code>a = [127, 128, 127]</code>. <code>b.subList(0, 2).clear()</code>: structural modification through the <code>subList</code> view removes indices <code>0-1</code>. <code>b = [127, 128]</code>.
16.	D	The exception <code>IOException</code> cannot possibly be thrown by the code in the <code>try</code> block, so it is unreachable code, and throws a compiler error.
17.	C	The <code>keySet</code> of a <code>TreeMap</code> will be in sorted order and will not contain duplicates.
18.	D	The possible value range is <code>[0,98]</code> .
19.	A	The runtime of <code>TreeMap.put()</code> and <code>get()</code> is $\mathcal{O}(\log N)$, and we run it N times both times, so the runtime is $\mathcal{O}(N \log N)$
20.	D	No compile errors, and first runtime error is line <code>//4</code> with accessing an undefined array in position <code>2</code> .
21.	E	One of the conversions fails by trying to read a <code>7</code> as base <code>7</code> , so there is a runtime error.
22.	A	<code>Fighter</code> is a class, so must be extend-ed not implement-ed. Since both classes are involved it can only extend <code>Fighter</code> , not the <code>Wizard</code> or <code>Warrior</code> .
23.	E	Can only call the <code>super</code> constructor with no parameters and must initialize <code>magic</code> inside the constructor or it is <code>0</code> , so either <code>A</code> or <code>B</code> will work.
24.	C	The <code>f</code> object's health will get <code>10</code> deducted from the <code>w2</code> attack, and <code>20</code> deducted from the <code>w1</code> attack.
25.	B	Nothing is ever deducted from the <code>w1</code> object's health, and it attacks twice so the <code>magic</code> variable will be <code>2</code> lower (<code>25 $\rightarrow$ 23</code>).
26.	D	The methods in the <code>Fighter</code> class are coupled to its <code>private health</code> instance variable, so any <code>set()</code> s will only work on that <code>health</code> variable, however, our <code>get()</code> to print out will grab the <code>health</code> variable from the <code>Warrior</code> class. The <code>heal()</code> method is different, as it is pointed to the <code>Warrior</code> instance variable <code>health</code> . So nothing will ever be subtracted, but the add from <code>heal()</code> will work, and since it is in the <code>Warrior</code> class, it begins at <code>300</code> .
27.	D	Complicated regex pattern matching (use regexr.com to check). The first pattern is simpler, using capture groups, boolean logic, and number quantifiers. The second pattern is using negative and positive look behind assertions.
28.	C	Simple Recursive Tracing.
29.	D	Simple Recursive Tracing.
30.	D	TIMSORT is a combination of INSERTIONSORT and MERGESORT.
31.	D	https://introcs.cs.princeton.edu/java/11precedence/ https://docs.oracle.com/javase/specs/jls/se22/html/jls-15.html#jls-15.15 The Java Language Specification defines associativity through its grammar. In §15.15, unary expressions are described as grouping right-to-left (e.g., <code>--x</code> is parsed as <code>-(~x)</code>), which applies to prefix unary operators such as <code>++x</code> and <code>--x</code>. However, the grammar also separates postfix expressions (<code>PostfixExpression</code>) from unary expressions. As a result, postfix operators like <code>x++</code> and <code>x--</code> associate left-to-right, while prefix operators remain right-to-left.
32.	B	The stream starts at <code>ABC</code> and rotates left each step: <code>ABC, BCA, CAB, ABC, BCA</code>. The predicate <code>s >= BCA</code> is true for <code>BCA</code> and <code>CAB</code>, so <code>negate()</code> keeps only <code>ABC</code> values. Mapping to first two characters gives <code>AB</code> and <code>AB</code>, and <code>joining()</code> produces <code>ABAB</code>.

33.	A	The method <code>getDescription()</code> returns the <code>String</code> of description, which is initialized via the constructor. Similarly, <code>getLevel()</code> returns the <code>int</code> of the level, which is also initialized via the constructor. <code>isHarderThan()</code> does a standard greater-than comparison and functions as expected. <code>fromLevel()</code> returns the <code>Difficulty</code> enum Object which has level equal to the passed <code>int</code>. If no such matching level exists, <code>null</code> is returned instead. So, we get "Beginner" as the description of <code>Difficulty.EASY</code>, <code>true</code> since <code>3 > 1</code>, "MEDIUM" since printing an enum without a <code>toString()</code> override prints the name of the enum's value, and <code>null</code> for the last since the enum value does not exist.
34.	C	Enums can have constructors and their fields do not need to be declared as static. By default, the <code>values()</code> method for an enum returns an array containing all of the constants of that enum type in the order they were declared, and thus, does not require manual implementation. This leaves whether the constants alone are instances of the owning enum type. This fact can be confirmed by making use of the <code>instanceof</code> Java operator, which will evaluate to <code>true</code> for all constants returned in the <code>values()</code> method of a given enum.
35.	B	Note that enum constants do not need to be assigned to a variable before you can access their methods. This is because each of the enum constants are by definition <code>static</code> and <code>final</code>, and thus, we are calling the operation on the global copy of the enum and not a locally-defined one (much the same way you can make use of <code>Integer.MAX_VALUE</code> and call its methods without assigning it to a variable).
36.	B	In a try-with-resources block, resources are closed in reverse declaration order. Construction prints <code>+A</code> then <code>+B</code>, body prints <code>X</code>, then <code>close()</code> prints <code>-B</code> then <code>-A</code>. Final output: <code>+A+BX-B-A</code>.
37.	C	Reference: https://personal.utdallas.edu/~daescu/maxsa.pdf This is a divide and conquer algorithm to determine the maximum subarray sum in $\mathcal{O}(n)$ time.
38.	A	See above.
39.	13	The following is an example of an MST of graph G, which has minimum weight of 13. 
40.	2	The following are the two unique traversals of the graph which have a minimum cost of 13. 